ALGORITMIA

Grado en Ingeniería Informática del Software - Curso 2024-2025

GUIÓN DE LA PRÁCTICA 4 (directorio p4)

0) INTRODUCCIÓN

El coloreado de mapas es uno de los casos de aplicación del algoritmo devorador o ávido. Dado un mapa con n regiones, el objetivo consiste en asignar a cada región un color distinto al de todas las regiones adyacentes utilizando el menor número de colores posible.

En este caso, el mapa será representado por un grafo construido sobre una cuadrícula, en el cual cada nodo representa una región del mapa y las conexiones entre nodos representan fronteras entre regiones. Un nodo puede tener hasta 8 conexiones con otros nodos, simbolizando las direcciones cardinales y ordinales.

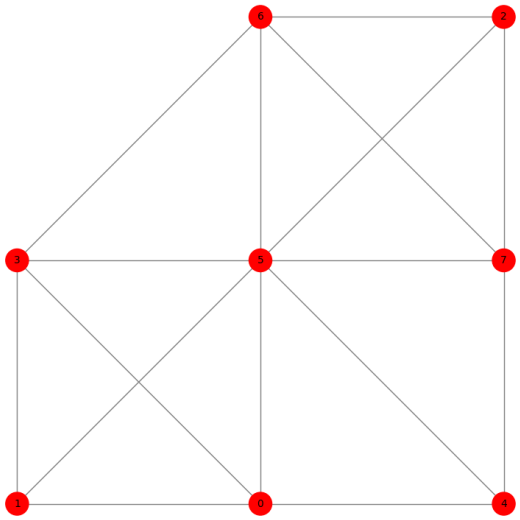

Ilustración 1 - Visualización del grafo g8.json con la función dibujar_mapa_coloreado

1)COLOREADO DE MAPAS

Se le proporciona el módulo **auxiliar.py**, que tiene dos funciones:

- *Generar_mapa_grafo* construye un grafo que posee las características descritas con el nº de nodos indicado. Retorna un JSON con dos valores: "*nodos*" contiene las posiciones de cada nodo en la cuadrícula (exclusivamente para construir la visualización) y "*grafo*" contiene, para cada nodo, una lista de sus nodos vecinos. Asimismo, dicho JSON es guardado en un archivo "*grafo.json*".
- *Dibujar_mapa_coloreado* recibe como parámetros un grafo en formato JSON generado por la función anterior y un JSON "*colores*" que debe contener, para cada nodo, el color que le corresponde.

Para su correcto funcionamiento, ejecútese en el directorio del proyecto el comando **pip install -r requirements.txt** , que instalará las librerías necesarias.

Este módulo incluye adicionalmente un ejemplo de uso en el que se genera un grafo de 128 nodos y una visualización con colores homogéneos.

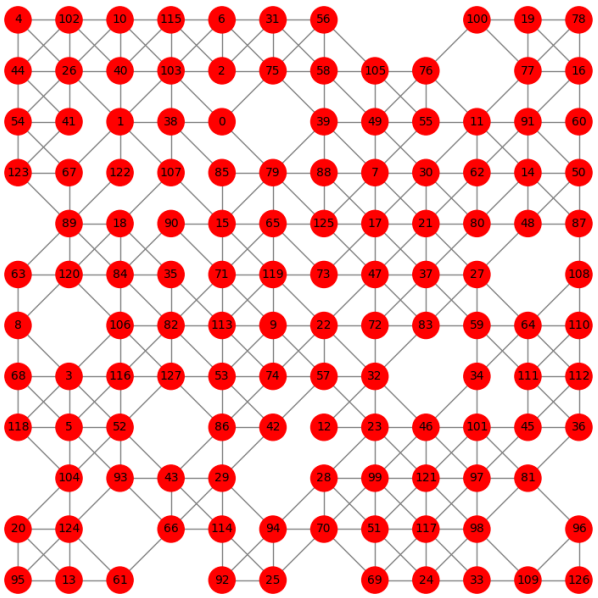

Ilustración 2 - Visualización del grafo g128.json con colores homogéneos

El objetivo de esta práctica consiste en resolver, mediante la implementación de un algoritmo devorador, el coloreado de estos grafos acorde a las restricciones descritas. Por ejemplo, para el anterior, una posible solución, con 6 colores, sería:

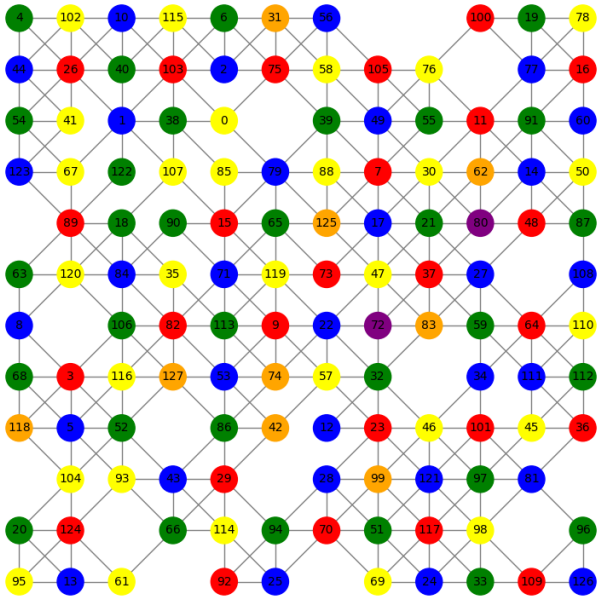

Ilustración 3 - Visualización del grafo g128.json con los colores de la solución s128.json

En el directorio **sols** se proporcionan como referencia posibles soluciones, calculadas con un determinado heurístico, para grafos de tamaño variado. En el caso de que hubiere varias soluciones óptimas con el mismo nº de colores empleado nos vale cualquiera de ellas, por lo que podría haber otras soluciones válidas a las propuestas, pero evidentemente deberían emplear el mismo nº de colores, o menor, si se define un heurístico mejor.

Un listado de colores propuesto para la asignación es *["red", "blue", "green", "yellow", "orange", "purple", "cyan", "magenta", "lime"].*

SE LE PIDE:

Según lo que indique su profesor de prácticas:

Implementar una clase ColoreoGrafo.java, de modo que con la clase proporcionada Devorador.java podamos calcular una solución y visualizarla con el módulo proporcionado en Python. Se debe añadir el JAR provisto al *build path* para que este último funcione. **Razone convenientemente la complejidad temporal del algoritmo implementado. Implementar una clase DevoradorTiempos.java, que emplee los grafos contenidos en sols y calculando el tiempo que tarda el algoritmo hecho en el apartado anterior en resolver el problema, de forma que se pueda ir rellenando la siguiente tabla.** **Razone: ¿siguen la complejidad calculada los tiempos de la tabla?**
Implementar el módulo coloreado_grafo.py, de modo que podamos calcular una solución y visualizarla con el módulo proporcionado en Python. **Razone convenientemente la complejidad temporal del algoritmo implementado. Implementar un módulo devorador_tiempos.py, que emplee los grafos contenidos en sols y calculando el tiempo que tarda el algoritmo hecho en el apartado anterior en resolver el problema, de forma que se pueda ir rellenando la siguiente tabla.** **Razone: ¿siguen la complejidad calculada los tiempos de la tabla?**

TABLA TIEMPOS ALGORITMO COLOREADO

n	***t Coloreado (ms)***
8	
16	
32	
…	
8192	
16384	
32768	
65536	

Se ha de entregar en un **.pdf** el trabajo que se le pide y además los módulos/clases **.py/.java** que ha programado. Todo ello lo organizará en una carpeta, que es la que entregará comprimida en un fichero **practica4ApellidosNombre.zip**.

La entrega de esta práctica se realizará, en tiempo y forma, según las indicaciones dadas por el profesor de práctica.